

Sidetick — PRD, Technical Architecture & Workflow Documentation

This document defines the complete Product Requirements Document (PRD), technical architecture, workflows, security strategy, infrastructure stack, and deployment blueprint for the Sidetick trading education platform. The platform is inspired by modern quantitative education ecosystems such as QuantInsti and tailored for Indian retail traders.

1. Executive Summary

Sidetick is a premium algorithmic trading education platform focused on TradingView automation, Pine Script workflows, Dhan API integrations, and systematic trading education. The platform aims to convert warm traffic from Instagram, WhatsApp, and Telegram into paid learners through high-converting landing pages, gated content systems, secure streaming, and automated onboarding.

2. Core Objectives

- Convert social traffic into paid students
 - Deliver secure premium course content
 - Prevent video piracy and credential sharing
 - Automate user onboarding after payment
 - Build scalable infrastructure for future expansion
 - Create a QuantInsti-style ecosystem for trading education
-

3. User Personas

Retail Trader: Wants to automate strategies and reduce emotional trading.

Semi-Technical Trader: Interested in APIs, automation, and deployment.

Warm Community Lead: Already follows Sidetick through social channels and needs trust-building before purchase.

4. Platform Modules

Public Website: Landing pages, sales funnels, testimonials, pricing, FAQs.

Member Portal: Protected learning dashboard with streaming videos and progress tracking.

Authentication: OTP login, passwordless authentication, secure session handling.

Payments: Razorpay integration with automated access provisioning.

Admin Dashboard: Manage courses, users, analytics, and content delivery.

Lead Magnet System: Freebies page for WhatsApp lead generation.

5. Recommended Technical Stack

Frontend: Next.js 15, TailwindCSS, shadcn/ui, Framer Motion

Backend: Node.js, Express, Prisma ORM

Database: PostgreSQL

Authentication: Clerk or Supabase Auth

Video Streaming: Bunny Stream with tokenized HLS delivery

Payments: Razorpay

Infrastructure: Vercel, Railway, Cloudflare CDN

Analytics: PostHog, Google Analytics, Meta Pixel

6. Website Information Architecture

Homepage → Course Sales Page → Razorpay Checkout → Member Dashboard

Additional Pages:

- Freebies Page
 - Login / OTP Authentication
 - Dashboard
 - Account Settings
 - Admin Panel
-

7. Razorpay Workflow

User clicks “Buy Now” → Razorpay Checkout → Payment Success → Webhook Validation → User Account Creation → Course Access Granted → Login Link Sent → Redirect to Dashboard.

8. Video Streaming & Piracy Protection

Piracy protection is the most critical system requirement.

Security layers include:

- Signed temporary playback URLs
- HLS encrypted chunk streaming
- JWT token validation
- Session restrictions
- Dynamic email watermark overlays
- Referer protection
- Concurrent device limitations

9. Database Design

Core tables include:

- Users
- Purchases
- Courses
- Modules
- Lessons
- Progress Tracking
- Sessions
- Freebie Downloads

10. Authentication System

Passwordless authentication using OTP login is recommended for Indian users due to lower friction and higher conversion rates. Supported flows:

- WhatsApp OTP
- Mobile OTP
- Email magic links

11. Progress Tracking

The system tracks:

- Watch percentage
- Lesson completion
- Last watched timestamp
- Resume playback position
- Time spent learning

12. Admin Dashboard Requirements

Admin features include:

- User management
- Course creation
- Video upload management
- Revenue analytics
- Access revocation
- Coupon management
- Engagement metrics

13. Security Requirements

- HTTPS enforced across all services
 - JWT authentication
 - Rate limiting
 - SQL injection prevention
 - Webhook signature validation
 - Secure session storage
 - CDN-level protection through Cloudflare
-

14. Infrastructure Architecture

Frontend hosted on Vercel.
Backend services hosted on Railway or Render.
PostgreSQL database for persistent storage.
Bunny Stream for secure video delivery.
Cloudflare CDN for caching and DDoS protection.

15. Scalability Strategy

Phase 1: MVP platform for course delivery.
Phase 2: Blog, webinars, certificates, quizzes.
Phase 3: AI trading assistant, backtesting engine, broker integrations, mobile applications.

16. SEO & Marketing

Inspired by QuantInsti's SEO-heavy acquisition strategy.

Future content strategy includes:

- Pine Script tutorials
 - TradingView automation guides
 - Dhan API tutorials
 - Strategy breakdowns
 - Case studies
 - Webinar funnels
-

17. Development Timeline

UI/UX Design: 4–5 days
Frontend Development: 7–10 days
Backend APIs: 5–7 days
Video Security: 3–4 days
Payment Integration: 2 days
QA & Deployment: 3–4 days

Estimated production-ready MVP timeline: 3–4 weeks.

18. Final Technical Recommendation

Recommended architecture:

Next.js + PostgreSQL + Razorpay + Bunny Stream + Cloudflare.

This stack provides:

- High performance
- Strong piracy protection
- Scalability
- Secure payments
- Modern UX
- Mobile responsiveness
- Future expansion flexibility

References & Inspiration

- QuantInsti EPAT Platform Architecture
- QuantInsti Quantra Learning System
- Modern LMS and SaaS Architecture Patterns
- Razorpay Payment Automation Flows
- Bunny Stream DRM and HLS Streaming Architecture